In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [2]:
```
# 55 to 58
data = {
    "name": [
        "William", "Emma", "Sofia", "Markus", "Edward",
        "Thomas", "Ethan", np.nan, "Arun", "Anika", "Paulo"
    ],
    "region": [
        np.nan, "North", "East", np.nan, "West",
        "West", "South", np.nan, "West", "East", "South"
    ],
    "sales": [
        50000.0, 52000.0, np.nan, np.nan, 42000.0,
        72000.0, 49000.0, np.nan, 67000.0, 65000.0, 67000.0
    ],
    "expenses": [
        42000.0, 43000.0, np.nan, np.nan, 38000.0,
        39000.0, 42000.0, np.nan, 39000.0, 50000.0, 45000.0
    ]
}

df = pd.DataFrame(data)
```

In [3]:
```
# 55

df.dropna()
```

Out[3]:

	name	region	sales	expenses
1	Emma	North	52000.0	43000.0
4	Edward	West	42000.0	38000.0
5	Thomas	West	72000.0	39000.0
6	Ethan	South	49000.0	42000.0
8	Arun	West	67000.0	39000.0
9	Anika	East	65000.0	50000.0
10	Paulo	South	67000.0	45000.0

In [4]:
```
# 56

df.dropna(thresh=1)
```

Out[4]:

	name	region	sales	expenses
0	William	NaN	50000.0	42000.0
1	Emma	North	52000.0	43000.0
2	Sofia	East	NaN	NaN
3	Markus	NaN	NaN	NaN
4	Edward	West	42000.0	38000.0
5	Thomas	West	72000.0	39000.0
6	Ethan	South	49000.0	42000.0
8	Arun	West	67000.0	39000.0
9	Anika	East	65000.0	50000.0
10	Paulo	South	67000.0	45000.0

In [5]:

```
# 57

df.dropna(axis=1)
```

Out[5]:

0
1
2
3
4
5
6
7
8
9
10

In [6]:

```
# 58

def remove_outliers(x):
    q1 = x.quantile(0.25)
    q3 = x.quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    print(f"Lower bound: {lower_bound}, Upper bound: {upper_bound}")
    return x[(x >= lower_bound) & (x <= upper_bound)]

remove_outliers(df["expenses"])
```

```
Lower bound: 32250.0, Upper bound: 50250.0
```

Out[6]:
```
0     42000.0
1     43000.0
4     38000.0
5     39000.0
6     42000.0
8     39000.0
9     50000.0
10    45000.0
Name: expenses, dtype: float64
```

In [7]:
```python
# 59

data = {
    "A": ["TeamA", "TeamB", "TeamB", "TeamC", "TeamA"],
    "B": [50, 40, 40, 30, 50],
    "C": [True, False, False, False, True]
}
df = pd.DataFrame(data)
print(df)
df.drop_duplicates()
```

```
      A   B      C
0  TeamA  50   True
1  TeamB  40  False
2  TeamB  40  False
3  TeamC  30  False
4  TeamA  50   True
```

Out[7]:

	A	B	C
0	TeamA	50	True
1	TeamB	40	False
3	TeamC	30	False

In [8]:
```python
# 60

df =pd.read_csv("https://raw.githubusercontent.com/Jovita7/Data-Analysis-and-Vis

df.drop(columns=["mpg","cylinders"])
```

Out[8]:

	displacement	horsepower	weight	acceleration	model year	origin	car name
0	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	350.0	165	3693	11.5	70	1	buick skylark 320
2	318.0	150	3436	11.0	70	1	plymouth satellite
3	304.0	150	3433	12.0	70	1	amc rebel sst
4	302.0	140	3449	10.5	70	1	ford torino
...	...	...	...	...	...	...	...
393	140.0	86	2790	15.6	82	1	ford mustang gl
394	97.0	52	2130	24.6	82	2	vw pickup
395	135.0	84	2295	11.6	82	1	dodge rampage
396	120.0	79	2625	18.6	82	1	ford ranger
397	119.0	82	2720	19.4	82	1	chevy s-10

398 rows × 7 columns

In [9]:

```
# 61
df =pd.read_csv("https://raw.githubusercontent.com/Jovita7/Data-Analysis-and-Vis
df
```

Out[9]:

	Height	Weight	Male
0	73.847017	241.893563	1
1	68.781904	162.310473	1
2	74.110105	212.740856	1
3	71.730978	220.042470	1
4	69.881796	206.349801	1
...	...	...	...
9995	66.172652	136.777454	0
9996	67.067155	170.867906	0
9997	63.867992	128.475319	0
9998	69.034243	163.852461	0
9999	61.944246	113.649103	0

10000 rows × 3 columns

In [10]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Height  10000 non-null  float64
 1   Weight  10000 non-null  float64
 2   Male    10000 non-null  int64
dtypes: float64(2), int64(1)
memory usage: 234.5 KB
```

In [11]:
```
df.describe()
```

Out[11]:

	Height	Weight	Male
count	10000.000000	10000.000000	10000.000000
mean	66.367560	161.440357	0.500000
std	3.847528	32.108439	0.500025
min	54.263133	64.700127	0.000000
25%	63.505620	135.818051	0.000000
50%	66.318070	161.212928	0.500000
75%	69.174262	187.169525	1.000000
max	78.998742	269.989699	1.000000

In [12]:
```
df.corr()
```

Out[12]:

	Height	Weight	Male
Height	1.000000	0.924756	0.691072
Weight	0.924756	1.000000	0.796723
Male	0.691072	0.796723	1.000000

In [13]:
```
remove_outliers(df["Weight"])
```

```
Lower bound: 58.79084096072856, Upper bound: 264.19673521345646
```

Out[13]:
```
0       241.893563
1       162.310473
2       212.740856
3       220.042470
4       206.349801
           ...
9995    136.777454
9996    170.867906
9997    128.475319
9998    163.852461
9999    113.649103
Name: Weight, Length: 9999, dtype: float64
```

In [14]:
```
remove_outliers(df["Height"])
```

```
Lower bound: 55.00265861279534, Upper bound: 77.67722359525831
```

Out[14]:
```
0       73.847017
1       68.781904
2       74.110105
3       71.730978
4       69.881796
          ...
9995    66.172652
9996    67.067155
9997    63.867992
9998    69.034243
9999    61.944246
Name: Height, Length: 9992, dtype: float64
```

In [15]:
```
# 62

df= pd.read_csv("../../datasets/ipl-matches.csv")
df
```

Out[15]:

	ID	City	Date	Season	MatchNumber	Team1	Team2	
0	1312200	Ahmedabad	2022-05-29	2022	Final	Rajasthan Royals	Gujarat Titans	Ah
1	1312199	Ahmedabad	2022-05-27	2022	Qualifier 2	Royal Challengers Bangalore	Rajasthan Royals	Ah
2	1312198	Kolkata	2022-05-25	2022	Eliminator	Royal Challengers Bangalore	Lucknow Super Giants	
3	1312197	Kolkata	2022-05-24	2022	Qualifier 1	Rajasthan Royals	Gujarat Titans	
4	1304116	Mumbai	2022-05-22	2022	70	Sunrisers Hyderabad	Punjab Kings	\
...	...	...	...	...	...	...	...	
945	335986	Kolkata	2008-04-20	2007/08	4	Kolkata Knight Riders	Deccan Chargers	
946	335985	Mumbai	2008-04-20	2007/08	5	Mumbai Indians	Royal Challengers Bangalore	\
947	335984	Delhi	2008-04-19	2007/08	3	Delhi Daredevils	Rajasthan Royals	F
948	335983	Chandigarh	2008-04-19	2007/08	2	Kings XI Punjab	Chennai Super Kings	A
949	335982	Bangalore	2008-04-18	2007/08	1	Royal Challengers Bangalore	Kolkata Knight Riders	Chi

950 rows × 20 columns

In [16]:
```
df[df["SuperOver"]=="Y"]
```

Out[16]:

	ID	City	Date	Season	MatchNumber	Team1	Team2	
114	1254077	Chennai	2021-04-25	2021	20	Delhi Capitals	Sunrisers Hyderabad	Chi
158	1216512	Abu Dhabi	2020-10-18	2020/21	35	Kolkata Knight Riders	Sunrisers Hyderabad	She
159	1216517	NaN	2020-10-18	2020/21	36	Mumbai Indians	Kings XI Punjab	Int
184	1216547	NaN	2020-09-28	2020/21	10	Royal Challengers Bangalore	Mumbai Indians	Int
192	1216493	NaN	2020-09-20	2020/21	2	Delhi Capitals	Kings XI Punjab	Int
203	1178426	Mumbai	2019-05-02	2019	51	Mumbai Indians	Sunrisers Hyderabad	
244	1175365	Delhi	2019-03-30	2019	10	Kolkata Knight Riders	Delhi Capitals	A
339	1082625	Rajkot	2017-04-29	2017	35	Gujarat Lions	Mumbai Indians	A
474	829741	Ahmedabad	2015-04-21	2015	18	Rajasthan Royals	Kings XI Punjab	Sa
533	729315	Abu Dhabi	2014-04-29	2014	19	Kolkata Knight Riders	Rajasthan Royals	She
608	598017	Bangalore	2013-04-16	2013	21	Royal Challengers Bangalore	Delhi Daredevils	Chi
621	598004	Hyderabad	2013-04-07	2013	7	Sunrisers Hyderabad	Royal Challengers Bangalore	Ra Int

	ID	City	Date	Season	MatchNumber	Team1	Team2	
819	419121	Chennai	2010-03-21	2009/10	16	Chennai Super Kings	Kings XI Punjab	Chi
883	392190	Cape Town	2009-04-23	2009	10	Kolkata Knight Riders	Rajasthan Royals	

In [17]:
```
df[(df["WinningTeam"]=="Chennai Super Kings") & (df["Venue"]=="Eden Gardens, Kol
```

Out[17]:

ID	City	Date	Season	MatchNumber	Team1	Team2	Venue	TossWinner	TossDeci

In [18]:
```
df[(df["TossWinner"]=="Gujarat Titans")& (df["TossDecision"]=="Bat") & (df["Winn
```

Out[18]:

ID	City	Date	Season	MatchNumber	Team1	Team2	Venue	TossWinner	TossDeci

In [19]:
```
df[df["WinningTeam"]=="Gujarat Titans"]
```

Out[19]:

	ID	City	Date	Season	MatchNumber	Team1	Team2	Ve
0	1312200	Ahmedabad	2022-05-29	2022	Final	Rajasthan Royals	Gujarat Titans	Nare M Stac Ahmeda
3	1312197	Kolkata	2022-05-24	2022	Qualifier 1	Rajasthan Royals	Gujarat Titans	Garc Ko
12	1304108	Mumbai	2022-05-15	2022	62	Chennai Super Kings	Gujarat Titans	Wankl Stac Mu
17	1304103	Pune	2022-05-10	2022	57	Gujarat Titans	Lucknow Super Giants	Maharas Cr Associa Stac
31	1304089	Mumbai	2022-04-30	2022	43	Royal Challengers Bangalore	Gujarat Titans	Brabo Stac Mu
34	1304086	Mumbai	2022-04-27	2022	40	Sunrisers Hyderabad	Gujarat Titans	Wankl Stac Mu
39	1304081	Navi Mumbai	2022-04-23	2022	35	Gujarat Titans	Kolkata Knight Riders	Dr DY Sp Acad Mu
45	1304075	Pune	2022-04-17	2022	29	Chennai Super Kings	Gujarat Titans	Maharas Cr Associa Stac
50	1304070	Mumbai	2022-04-14	2022	24	Gujarat Titans	Rajasthan Royals	Dr DY Sp Acad Mu
58	1304062	Mumbai	2022-04-08	2022	16	Punjab Kings	Gujarat Titans	Brabo Stac Mu
64	1304056	Pune	2022-04-02	2022	10	Gujarat Titans	Delhi Capitals	Maharas Cr

	ID	City	Date	Season	MatchNumber	Team1	Team2	Ve
								Associ Stac
70	1304050	Mumbai	2022-03-28	2022	4	Lucknow Super Giants	Gujarat Titans	Wankl Stac Mu

In [20]:
```
# 64

df = pd.read_csv("../../datasets/car data.csv")
df
```

Out[20]:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer
4	swift	2014	4.60	6.87	42450	Diesel	Dealer
...	...	...	...	...	...	...	...
296	city	2016	9.50	11.60	33988	Diesel	Dealer
297	brio	2015	4.00	5.90	60000	Petrol	Dealer
298	city	2009	3.35	11.00	87934	Petrol	Dealer
299	city	2017	11.50	12.50	9000	Diesel	Dealer
300	brio	2016	5.30	5.90	5464	Petrol	Dealer

301 rows × 9 columns

In [21]:
```
df[(df["Kms_Driven"]>30000) & (df["Car_Name"]=="ritz")]
```

Out[21]:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type
30	ritz	2012	3.10	5.98	51439	Diesel	Dealer
31	ritz	2011	2.35	4.89	54200	Petrol	Dealer
46	ritz	2013	2.65	4.89	64532	Petrol	Dealer

In [22]:
```
df[(df["Year"]==2017) & (df["Selling_Price"]>10) & (df["Fuel_Type"]=="Petrol")]
```

Out[22]:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type
66	innova	2017	19.75	23.15	11000	Petrol	Dealer
97	corolla altis	2017	17.00	18.64	8700	Petrol	Dealer

In [23]:
```
df[(df["Car_Name"]=="swift") & (df["Selling_Price"]<4)]
```

Out[23]:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type
25	swift	2011	3.0	4.99	20000	Petrol	Dealer

In [24]:
```
df[(df["Year"]==2015) & (df["Selling_Price"]>10) & (df["Fuel_Type"]=="Petrol") &
```

Out[24]:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type
232	elantra	2015	11.45	14.79	12900	Petrol	Dealer

In [25]:
```
df[(df["Selling_Price"]<1) & (df["Transmission"]=="Automatic")]
```

Out[25]:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type
155	Honda Activa 4G	2017	0.48	0.51	4300	Petrol	Individual
159	Honda Activa 4G	2017	0.45	0.51	4000	Petrol	Individual
165	Activa 3g	2016	0.45	0.54	500	Petrol	Individual
173	Activa 4g	2017	0.40	0.51	1300	Petrol	Individual
177	Honda Activa 125	2016	0.35	0.57	24000	Petrol	Individual
178	TVS Jupyter	2014	0.35	0.52	19000	Petrol	Individual
185	Suzuki Access 125	2008	0.25	0.58	1900	Petrol	Individual
186	TVS Wego	2010	0.25	0.52	22000	Petrol	Individual
196	Activa 3g	2008	0.17	0.52	500000	Petrol	Individual

In [26]:
```
df[(df["Kms_Driven"]>50000) & (df["Fuel_Type"]=="Petrol")& (2010<=df["Year"]) &
```

Out[26]:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type
24	wagon r	2013	2.90	4.410	56879	Petrol	Dealer
26	swift	2013	4.15	5.870	55138	Petrol	Dealer
31	ritz	2011	2.35	4.890	54200	Petrol	Dealer
46	ritz	2013	2.65	4.890	64532	Petrol	Dealer
68	corolla altis	2011	4.35	13.740	88000	Petrol	Dealer
72	corolla altis	2013	7.45	18.610	56001	Petrol	Dealer
76	corolla altis	2013	5.50	14.680	72000	Petrol	Dealer
78	corolla altis	2010	5.25	22.830	80000	Petrol	Dealer
87	corolla altis	2012	5.90	13.740	56000	Petrol	Dealer
95	corolla altis	2012	5.85	18.610	72000	Petrol	Dealer
147	Yamaha FZ S V 2.0	2015	0.55	0.840	58000	Petrol	Individual
175	Hero Honda CBZ extreme	2011	0.38	0.787	75000	Petrol	Individual
179	Honda Karizma	2010	0.31	1.050	213000	Petrol	Individual
180	Hero Honda Passion Pro	2012	0.30	0.510	60000	Petrol	Individual
203	i10	2011	2.95	4.600	53460	Petrol	Dealer
227	i10	2011	2.55	4.430	57000	Petrol	Dealer
238	verna	2013	4.80	9.400	60241	Petrol	Dealer
251	city	2013	5.00	9.900	56701	Petrol	Dealer
254	city	2013	5.25	9.900	54242	Petrol	Dealer
255	brio	2012	3.00	5.350	53675	Petrol	Dealer
270	city	2011	4.10	10.000	69341	Petrol	Dealer
273	jazz	2010	2.25	7.500	61203	Petrol	Dealer
297	brio	2015	4.00	5.900	60000	Petrol	Dealer

In [27]:
```
df[df["Present_Price"]-df["Selling_Price"]>15]
```

Out[27]:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type
50	fortuner	2012	14.90	30.61	104707	Diesel	Dealer
59	fortuner	2014	19.99	35.96	41000	Diesel	Dealer
62	fortuner	2014	18.75	35.96	78000	Diesel	Dealer
78	corolla altis	2010	5.25	22.83	80000	Petrol	Dealer
79	fortuner	2012	14.50	30.61	89000	Diesel	Dealer
85	camry	2006	2.50	23.73	142000	Petrol	Individual
86	land cruiser	2010	35.00	92.60	78000	Diesel	Dealer
94	corolla altis	2008	4.00	22.78	89000	Petrol	Dealer

In [28]:
```
df[(df["Kms_Driven"]<5000) & (df["Fuel_Type"]=="Petrol")& (df["Selling_Price"]<0
```

Out[28]:

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type
155	Honda Activa 4G	2017	0.48	0.51	4300	Petrol	Individual
159	Honda Activa 4G	2017	0.45	0.51	4000	Petrol	Individual
165	Activa 3g	2016	0.45	0.54	500	Petrol	Individual
166	Hero Passion Pro	2016	0.45	0.55	1000	Petrol	Individual
173	Activa 4g	2017	0.40	0.51	1300	Petrol	Individual
185	Suzuki Access 125	2008	0.25	0.58	1900	Petrol	Individual

In [29]:
```
# 65

df = pd.read_csv("../../datasets/data_result.csv")
df
```

Out[29]:

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
0	8.11	7.68	7.11	7.43	8.18	115	Female	17020	16
1	6.48	5.90	4.15	4.29	4.96	115	Male	17021	16
2	8.41	8.24	7.52	8.25	7.75	115	Female	17022	16
3	7.33	6.83	6.33	6.79	6.89	115	Male	17023	16
4	7.89	7.34	7.22	7.32	7.46	115	Male	17024	16
5	7.33	6.72	6.48	6.50	7.39	115	Male	17025	16
6	7.15	6.72	6.89	6.86	7.64	115	Male	17026	16
7	6.74	6.38	4.93	5.64	6.75	115	Male	17027	16
8	7.89	7.45	6.96	7.71	8.00	115	Female	17028	16
9	7.30	6.34	6.30	6.61	6.18	115	Male	17029	16
10	8.00	7.90	7.52	7.64	8.11	115	Male	17030	16
11	6.78	4.28	5.81	6.04	6.39	115	Male	17031	16
12	5.67	5.62	5.48	5.93	6.75	115	Male	17032	16
13	6.33	6.41	6.41	6.93	7.00	115	Male	17033	16
14	6.44	6.21	6.19	6.68	6.86	115	Male	17034	16
15	6.30	6.17	5.59	5.89	6.36	115	Male	17035	16
16	6.96	6.14	6.93	7.18	7.29	115	Male	17036	16
17	6.70	6.45	5.78	6.00	6.61	115	Male	17037	16
18	7.41	6.93	7.33	7.71	8.46	115	Male	17038	16
19	7.04	6.31	6.44	7.21	7.32	115	Male	17039	16
20	6.63	5.62	6.11	5.79	6.25	115	Male	17040	16
21	8.33	8.72	7.81	8.04	8.93	115	Female	17041	16
22	7.96	7.93	7.78	8.21	8.21	115	Male	17042	16
23	6.89	5.55	5.93	6.79	7.54	115	Male	17043	16
24	8.19	8.17	7.15	7.75	7.64	115	Male	17044	16
25	7.89	6.69	7.15	8.43	8.36	115	Female	17045	16
26	7.33	6.55	5.96	6.75	6.89	115	Male	17046	16
27	8.22	8.17	8.37	9.04	9.00	115	Male	17047	16
28	7.44	7.59	7.26	7.89	7.86	115	Male	17048	16
29	5.74	6.45	6.48	7.29	7.61	115	Male	17049	16
30	6.81	6.79	6.30	7.14	7.86	115	Female	17050	16
31	8.89	8.31	7.30	9.25	8.50	115	Female	17051	16
32	8.04	7.59	7.22	8.32	8.82	115	Male	17052	16

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
33	8.30	8.38	7.63	7.93	8.75	115	Male	17053	16
34	8.04	6.69	6.37	6.64	6.64	115	Male	17054	16
35	7.44	7.97	7.78	8.64	8.71	115	Male	17055	16
36	7.85	6.90	7.04	7.82	8.39	115	Female	17056	16
37	8.07	7.34	7.59	7.89	8.14	115	Male	17057	16
38	7.59	7.38	7.30	8.64	8.21	115	Male	17058	16
39	8.19	7.66	7.48	8.68	8.89	115	Male	17059	16
40	5.81	5.83	4.15	4.93	4.86	115	Male	17060	16
41	6.30	6.24	5.85	6.36	7.00	115	Male	17061	16
42	7.78	6.93	7.44	7.86	8.21	115	Male	17062	16
43	8.22	6.66	7.07	7.29	7.61	115	Male	17063	16
44	7.67	7.07	7.04	7.07	6.93	115	Male	17064	16
45	8.41	7.59	7.41	7.89	8.11	115	Male	17065	16

In [30]:
```
df.head()
```

Out[30]:

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
0	8.11	7.68	7.11	7.43	8.18	115	Female	17020	16
1	6.48	5.90	4.15	4.29	4.96	115	Male	17021	16
2	8.41	8.24	7.52	8.25	7.75	115	Female	17022	16
3	7.33	6.83	6.33	6.79	6.89	115	Male	17023	16
4	7.89	7.34	7.22	7.32	7.46	115	Male	17024	16

In [31]:
```
df.shape
```

Out[31]:
```
(46, 9)
```

In [32]:
```
df.tail()
```

Out[32]:

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
41	6.30	6.24	5.85	6.36	7.00	115	Male	17061	16
42	7.78	6.93	7.44	7.86	8.21	115	Male	17062	16
43	8.22	6.66	7.07	7.29	7.61	115	Male	17063	16
44	7.67	7.07	7.04	7.07	6.93	115	Male	17064	16
45	8.41	7.59	7.41	7.89	8.11	115	Male	17065	16

In [33]:
```
df.describe()
```

Out[33]:

	1st	2nd	3rd	4th	5th	College Code	Roll no.	Su
count	46.000000	46.000000	46.000000	46.000000	46.000000	46.0	46.000000	
mean	7.397609	6.930217	6.703043	7.237826	7.527609	115.0	17042.500000	
std	0.798391	0.910425	0.917324	1.057981	0.967963	0.0	13.422618	
min	5.670000	4.280000	4.150000	4.290000	4.860000	115.0	17020.000000	
25%	6.787500	6.350000	6.217500	6.650000	6.890000	115.0	17031.250000	
50%	7.440000	6.810000	7.000000	7.290000	7.625000	115.0	17042.500000	
75%	8.040000	7.590000	7.322500	7.890000	8.210000	115.0	17053.750000	
max	8.890000	8.720000	8.370000	9.250000	9.000000	115.0	17065.000000	

In [34]:
```
df.describe(percentiles=[0.58,0.87])
```

Out[34]:

	1st	2nd	3rd	4th	5th	College Code	Roll no.	Su
count	46.000000	46.000000	46.000000	46.000000	46.000000	46.0	46.000000	
mean	7.397609	6.930217	6.703043	7.237826	7.527609	115.0	17042.500000	
std	0.798391	0.910425	0.917324	1.057981	0.967963	0.0	13.422618	
min	5.670000	4.280000	4.150000	4.290000	4.860000	115.0	17020.000000	
58%	7.787000	6.944000	7.114000	7.647000	7.860000	115.0	17046.100000	
87%	8.220000	8.000000	7.530500	8.336500	8.531500	115.0	17059.150000	
max	8.890000	8.720000	8.370000	9.250000	9.000000	115.0	17065.000000	

In [35]:
```
df.describe(include="all")
```

Out[35]:

	1st	2nd	3rd	4th	5th	College Code	Gender	R
count	46.000000	46.000000	46.000000	46.000000	46.000000	46.0	46	46.
unique	NaN	NaN	NaN	NaN	NaN	NaN	2	
top	NaN	NaN	NaN	NaN	NaN	NaN	Male	
freq	NaN	NaN	NaN	NaN	NaN	NaN	38	
mean	7.397609	6.930217	6.703043	7.237826	7.527609	115.0	NaN	17042.
std	0.798391	0.910425	0.917324	1.057981	0.967963	0.0	NaN	13.
min	5.670000	4.280000	4.150000	4.290000	4.860000	115.0	NaN	17020.
25%	6.787500	6.350000	6.217500	6.650000	6.890000	115.0	NaN	17031.
50%	7.440000	6.810000	7.000000	7.290000	7.625000	115.0	NaN	17042.
75%	8.040000	7.590000	7.322500	7.890000	8.210000	115.0	NaN	17053.
max	8.890000	8.720000	8.370000	9.250000	9.000000	115.0	NaN	17065.

In [36]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 46 entries, 0 to 45
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   1st           46 non-null     float64
 1   2nd           46 non-null     float64
 2   3rd           46 non-null     float64
 3   4th           46 non-null     float64
 4   5th           46 non-null     float64
 5   College Code  46 non-null     int64
 6   Gender        46 non-null     str
 7   Roll no.      46 non-null     int64
 8   Subject Code  46 non-null     int64
dtypes: float64(5), int64(3), str(1)
memory usage: 3.4 KB
```

In [37]:
```
df.isnull().sum()
```

Out[37]:
```
1st             0
2nd             0
3rd             0
4th             0
5th             0
College Code    0
Gender          0
Roll no.        0
Subject Code    0
dtype: int64
```

In [38]:
```
df.drop_duplicates()
```

Out[38]:

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
0	8.11	7.68	7.11	7.43	8.18	115	Female	17020	16
1	6.48	5.90	4.15	4.29	4.96	115	Male	17021	16
2	8.41	8.24	7.52	8.25	7.75	115	Female	17022	16
3	7.33	6.83	6.33	6.79	6.89	115	Male	17023	16
4	7.89	7.34	7.22	7.32	7.46	115	Male	17024	16
5	7.33	6.72	6.48	6.50	7.39	115	Male	17025	16
6	7.15	6.72	6.89	6.86	7.64	115	Male	17026	16
7	6.74	6.38	4.93	5.64	6.75	115	Male	17027	16
8	7.89	7.45	6.96	7.71	8.00	115	Female	17028	16
9	7.30	6.34	6.30	6.61	6.18	115	Male	17029	16
10	8.00	7.90	7.52	7.64	8.11	115	Male	17030	16
11	6.78	4.28	5.81	6.04	6.39	115	Male	17031	16
12	5.67	5.62	5.48	5.93	6.75	115	Male	17032	16
13	6.33	6.41	6.41	6.93	7.00	115	Male	17033	16
14	6.44	6.21	6.19	6.68	6.86	115	Male	17034	16
15	6.30	6.17	5.59	5.89	6.36	115	Male	17035	16
16	6.96	6.14	6.93	7.18	7.29	115	Male	17036	16
17	6.70	6.45	5.78	6.00	6.61	115	Male	17037	16
18	7.41	6.93	7.33	7.71	8.46	115	Male	17038	16
19	7.04	6.31	6.44	7.21	7.32	115	Male	17039	16
20	6.63	5.62	6.11	5.79	6.25	115	Male	17040	16
21	8.33	8.72	7.81	8.04	8.93	115	Female	17041	16
22	7.96	7.93	7.78	8.21	8.21	115	Male	17042	16
23	6.89	5.55	5.93	6.79	7.54	115	Male	17043	16
24	8.19	8.17	7.15	7.75	7.64	115	Male	17044	16
25	7.89	6.69	7.15	8.43	8.36	115	Female	17045	16
26	7.33	6.55	5.96	6.75	6.89	115	Male	17046	16
27	8.22	8.17	8.37	9.04	9.00	115	Male	17047	16
28	7.44	7.59	7.26	7.89	7.86	115	Male	17048	16
29	5.74	6.45	6.48	7.29	7.61	115	Male	17049	16
30	6.81	6.79	6.30	7.14	7.86	115	Female	17050	16
31	8.89	8.31	7.30	9.25	8.50	115	Female	17051	16
32	8.04	7.59	7.22	8.32	8.82	115	Male	17052	16

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
33	8.30	8.38	7.63	7.93	8.75	115	Male	17053	16
34	8.04	6.69	6.37	6.64	6.64	115	Male	17054	16
35	7.44	7.97	7.78	8.64	8.71	115	Male	17055	16
36	7.85	6.90	7.04	7.82	8.39	115	Female	17056	16
37	8.07	7.34	7.59	7.89	8.14	115	Male	17057	16
38	7.59	7.38	7.30	8.64	8.21	115	Male	17058	16
39	8.19	7.66	7.48	8.68	8.89	115	Male	17059	16
40	5.81	5.83	4.15	4.93	4.86	115	Male	17060	16
41	6.30	6.24	5.85	6.36	7.00	115	Male	17061	16
42	7.78	6.93	7.44	7.86	8.21	115	Male	17062	16
43	8.22	6.66	7.07	7.29	7.61	115	Male	17063	16
44	7.67	7.07	7.04	7.07	6.93	115	Male	17064	16
45	8.41	7.59	7.41	7.89	8.11	115	Male	17065	16

In [39]:
```
df[ (df["Gender"]=="Female") & (df["1st"]>7) & (df["2nd"]>7) & (df["3rd"]>7) & (
```

Out[39]:

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
0	8.11	7.68	7.11	7.43	8.18	115	Female	17020	16
2	8.41	8.24	7.52	8.25	7.75	115	Female	17022	16
21	8.33	8.72	7.81	8.04	8.93	115	Female	17041	16
31	8.89	8.31	7.30	9.25	8.50	115	Female	17051	16

In [40]:
```
df[(df["1st"]>8) & (df["2nd"]>8) & (df["3rd"]>8) & (df["4th"]>8) & (df["5th"]>8)
```

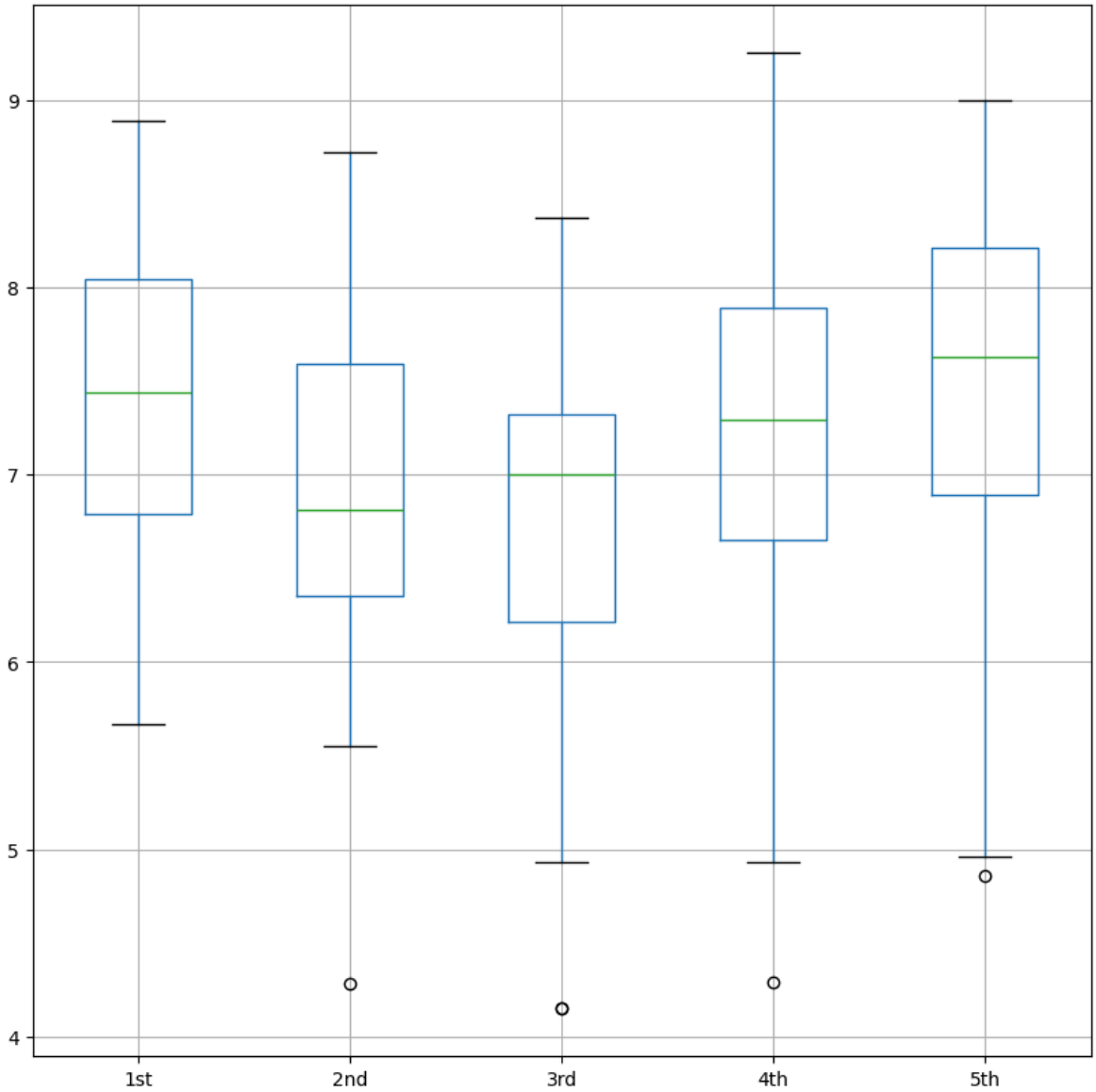
Out[40]:

	1st	2nd	3rd	4th	5th	College Code	Gender	Roll no.	Subject Code
27	8.22	8.17	8.37	9.04	9.0	115	Male	17047	16

In [41]:
```
def find_outliers(x):
    q1 = x.quantile(0.25)
    q3 = x.quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    print(f"Lower bound: {lower_bound}, Upper bound: {upper_bound}")
    return x[(x < lower_bound) | (x > upper_bound)]
```

In [42]:
```
find_outliers(df["4th"])
pd.plotting.boxplot(df[["1st", "2nd", "3rd", "4th", "5th"]], figsize=(10, 10))
plt.show()
```

```
Lower bound: 4.789999999999999, Upper bound: 9.75
```



In [43]:
```
# 66

df = pd.read_csv("../../datasets/movies.csv")
df
```

Out[43]:

	title_x	imdb_id	poster_path	
0	Uri: The Surgical Strike	tt8291224	https://upload.wikimedia.org/wikipedia/en/thum...	https://en.w
1	Battalion 609	tt9472208	NaN	https://(
2	The Accidental Prime Minister (film)	tt6986710	https://upload.wikimedia.org/wikipedia/en/thum...	https://en.wil
3	Why Cheat India	tt8108208	https://upload.wikimedia.org/wikipedia/en/thum...	https://en.v
4	Evening Shadows	tt6028796	NaN	https://en.w
...	...	...	...	
1624	Tera Mera Saath Rahen	tt0301250	https://upload.wikimedia.org/wikipedia/en/2/2b...	https://en.wik
1625	Yeh Zindagi Ka Safar	tt0298607	https://upload.wikimedia.org/wikipedia/en/thum...	https://en.wik
1626	Sabse Bada Sukh	tt0069204	NaN	https://en.w
1627	Daaka	tt10833860	https://upload.wikimedia.org/wikipedia/en/thum...	I
1628	Humsafar	tt2403201	https://upload.wikimedia.org/wikipedia/en/thum...	http

1629 rows × 18 columns

In [44]:
```
df[df["year_of_release"]==2019]
```

Out[44]:

	title_x	imdb_id	poster_path	
0	Uri: The Surgical Strike	tt8291224	https://upload.wikimedia.org/wikipedia/en/thum...	https://en
1	Battalion 609	tt9472208	NaN	https:
2	The Accidental Prime Minister (film)	tt6986710	https://upload.wikimedia.org/wikipedia/en/thum...	https://en.
3	Why Cheat India	tt8108208	https://upload.wikimedia.org/wikipedia/en/thum...	https://e
6	Fraud Saiyaan	tt5013008	https://upload.wikimedia.org/wikipedia/en/thum...	https:
...	...	...	...	
76	Commando 3 (film)	tt8983168	https://upload.wikimedia.org/wikipedia/en/thum...	https://en.w
77	Mardaani 2	tt5668770	https://upload.wikimedia.org/wikipedia/en/thum...	http
78	Dabangg 3	tt7059844	https://upload.wikimedia.org/wikipedia/en/thum...	htt
79	Good Newwz	tt8504014	NaN	https:
1627	Daaka	tt10833860	https://upload.wikimedia.org/wikipedia/en/thum...	

title_x	imdb_id	poster_path

75 rows × 18 columns

In [45]:
```
df[df["imdb_rating"]>7].shape[0]
```

Out[45]:
```
331
```

In [46]:
```
df[["title_x", "story"]][df["imdb_votes"]>20000]
```

Out[46]:

	title_x	story
0	Uri: The Surgical Strike	Divided over five chapters the film chronicle...
11	Gully Boy	Gully Boy is a film about a 22-year-old boy "M...
36	Kabir Singh	This Sandeep Vanga directorial is a remake of ...
74	Dil Bechara	A love story about two cancer patients.
88	Padmaavat	This fictional story is set in 13th century me...
...	...	...
1490	Devdas (2002 Hindi film)	Devdas Mukherji is black-listed by his multi-m...
1565	Kabhi Khushi Kabhie Gham...	Yashvardhan Raichand lives a very wealthy life...
1567	Lagaan	This is the story about the resilience shown b...
1568	Lagaan	This is the story about the resilience shown b...
1571	Dil Chahta Hai	Three young men Akash Sameer and Siddharth a...

105 rows × 2 columns

In [47]:
```
df[["title_x","year_of_release"]][df["year_of_release"]==2018]
```

Out[47]:

	title_x	year_of_release
4	Evening Shadows	2018
5	Soni (film)	2018
16	Mard Ko Dard Nahi Hota	2018
17	Hamid (film)	2018
20	Mere Pyare Prime Minister	2018
...	...	...
156	Rajma Chawal	2018
157	Zero (2018 film)	2018
158	Simmba	2018
166	Thugs of Hindostan	2018
1626	Sabse Bada Sukh	2018

79 rows × 2 columns

In [48]:
```
df[["title_x","wiki_link"]]
```

Out[48]:

	title_x	wiki_link
0	Uri: The Surgical Strike	https://en.wikipedia.org/wiki/Uri:_The_Surgica...
1	Battalion 609	https://en.wikipedia.org/wiki/Battalion_609
2	The Accidental Prime Minister (film)	https://en.wikipedia.org/wiki/The_Accidental_P...
3	Why Cheat India	https://en.wikipedia.org/wiki/Why_Cheat_India
4	Evening Shadows	https://en.wikipedia.org/wiki/Evening_Shadows
...	...	...
1624	Tera Mera Saath Rahen	https://en.wikipedia.org/wiki/Tera_Mera_Saath_...
1625	Yeh Zindagi Ka Safar	https://en.wikipedia.org/wiki/Yeh_Zindagi_Ka_S...
1626	Sabse Bada Sukh	https://en.wikipedia.org/wiki/Sabse_Bada_Sukh
1627	Daaka	https://en.wikipedia.org/wiki/Daaka
1628	Humsafar	https://en.wikipedia.org/wiki/Humsafar

1629 rows × 2 columns

In [49]:
```
# 67 same as previous question with different conditions
```

In [50]:
```
# 68

data = {
    "EmpID":[101,102,103,104,105],
    "Dept":["IT","HR","IT","Finance","HR"],
    "Salary":[60000,45000,80000,75000,50000],
```

```
    "Age":[25,32,29,41,28]
}
df = pd.DataFrame(data)
df
```

Out[50]:

	EmpID	Dept	Salary	Age
0	101	IT	60000	25
1	102	HR	45000	32
2	103	IT	80000	29
3	104	Finance	75000	41
4	105	HR	50000	28

In [51]:

```
df.sort_values(by=["Salary","Dept"], ascending=[False,True], inplace=True)
df
```

Out[51]:

	EmpID	Dept	Salary	Age
2	103	IT	80000	29
3	104	Finance	75000	41
0	101	IT	60000	25
4	105	HR	50000	28
1	102	HR	45000	32

In [52]:

```
df.sort_index(ascending=False)
```

Out[52]:

	EmpID	Dept	Salary	Age
4	105	HR	50000	28
3	104	Finance	75000	41
2	103	IT	80000	29
1	102	HR	45000	32
0	101	IT	60000	25

In [53]:

```
# 69

df = pd.DataFrame({
    "Student":["A","B","C","D"],
    "Math":[70,45,90,60],
    "Science":[75,40,85,55]
})
df
```

Out[53]:

	Student	Math	Science
0	A	70	75
1	B	45	40
2	C	90	85
3	D	60	55

In [54]:
```
df["Average"] = df.apply(lambda x: (x["Math"] + x["Science"])/2, axis=1)
df
```

Out[54]:

	Student	Math	Science	Average
0	A	70	75	72.5
1	B	45	40	42.5
2	C	90	85	87.5
3	D	60	55	57.5

In [55]:
```
df["grade"] = df.apply(lambda x: "Distinction" if x["Average"]>=80 else ("First"
df
```

Out[55]:

	Student	Math	Science	Average	grade
0	A	70	75	72.5	First
1	B	45	40	42.5	Second
2	C	90	85	87.5	Distinction
3	D	60	55	57.5	Second

In [56]:
```
# 70

df = pd.DataFrame({
    "Marks":["50","60","Absent","70","Absent"]
})
df
```

Out[56]:

	Marks
0	50
1	60
2	Absent
3	70
4	Absent

In [57]:
```
df.replace("Absent", 0, inplace=True)
df
```

Out[57]:

	Marks
0	50
1	60
2	0
3	70
4	0

In [58]:
```
df =df.astype({"Marks": int})
```

In [59]:
```
df.mean()
```

Out[59]:
```
Marks    36.0
dtype: float64
```

In [60]:
```
df[df["Marks"]>50]
```

Out[60]:

	Marks
1	60
3	70

In [61]:
```
# 71

df1 = pd.DataFrame({
    "ID":[1,2,3],
    "Name":["A","B","C"]
})
df2 = pd.DataFrame({
    "ID":[2,3,4],
    "Marks":[80,90,70]
})
df =df1.merge(df2, on="ID", how="outer")
```

In [62]:
```
df.isnull().sum()
```

Out[62]:
```
ID       0
Name     1
Marks    1
dtype: int64
```

In [63]:
```
df.sort_values(by="ID")
```

Out[63]:

	ID	Name	Marks
0	1	A	NaN
1	2	B	80.0
2	3	C	90.0
3	4	NaN	70.0

In [64]:
```
# 72

students = pd.DataFrame({
    "ID":[1,2,3,4],
    "Name":["A","B","C","D"]
})
marks = pd.DataFrame({
    "ID":[2,3,5],
    "Score":[85,90,75]
})

df = students.merge(marks, how="outer",indicator=True)
df
```

Out[64]:

	ID	Name	Score	_merge
0	1	A	NaN	left_only
1	2	B	85.0	both
2	3	C	90.0	both
3	4	D	NaN	left_only
4	5	NaN	75.0	right_only

In [65]:
```
# 73

df = pd.DataFrame({
    "Dept":["IT","IT","HR","HR","Finance","Finance"],
    "Salary":[50000,70000,40000,60000,80000,75000]
})

df.groupby("Dept")["Salary"].agg("mean")
```

Out[65]:
```
Dept
Finance    77500.0
HR         50000.0
IT         60000.0
Name: Salary, dtype: float64
```

In [66]:
```
df.groupby("Dept")["Salary"].agg("median")
```

Out[66]:
```
Dept
Finance    77500.0
HR         50000.0
IT         60000.0
Name: Salary, dtype: float64
```

In [67]:
```
df.groupby("Dept")["Salary"].agg("max")
```

Out[67]:
```
Dept
Finance    80000
HR         60000
IT         70000
Name: Salary, dtype: int64
```

In [68]:
```
# 74
```

```
df = pd.DataFrame({
    "Team":["A","A","A","B","B"],
    "Runs":[50,70,80,40,60]
})
df.groupby("Team")["Runs"].apply(lambda x: x.max() - x.min())
```

Out[68]:
```
Team
A    30
B    20
Name: Runs, dtype: int64
```

In [69]:
```
# 75

df = pd.DataFrame({
    "Salary":[20000,22000,21000,25000,24000,300000]
})
df=remove_outliers(df["Salary"])
df
```

```
Lower bound: 16000.0, Upper bound: 30000.0
```

Out[69]:
```
0    20000
1    22000
2    21000
3    25000
4    24000
Name: Salary, dtype: int64
```

In [70]:
```
df = pd.DataFrame({
    "Dept":["IT","IT","HR","HR","Finance","Finance"],
    "Employee":["A","B","C","D","E","F"],
    "City":["X","X","Y","Z","X","Y"]
})
df.groupby('Dept')['City'].nunique()
```

Out[70]:
```
Dept
Finance    2
HR         2
IT         1
Name: City, dtype: int64
```

In [71]:
```
df.groupby('Dept').sample(1)
```

Out[71]:

	Dept	Employee	City
4	Finance	E	X
3	HR	D	Z
0	IT	A	X

In [72]:
```
# 76

df = pd.DataFrame({
"Product":["P1","P2","P3","P4","P5"],
"Price":[100,200,150,300,250],
"Quantity":[5,2,4,1,3]
})
```

```
df["TotalAmount"] = df.apply(lambda x: x["Price"] * x["Quantity"], axis=1)
df
```

Out[72]:

	Product	Price	Quantity	TotalAmount
0	P1	100	5	500
1	P2	200	2	400
2	P3	150	4	600
3	P4	300	1	300
4	P5	250	3	750

In [73]:

```
df.sort_values("TotalAmount")
```

Out[73]:

	Product	Price	Quantity	TotalAmount
3	P4	300	1	300
1	P2	200	2	400
0	P1	100	5	500
2	P3	150	4	600
4	P5	250	3	750

In [74]:

```
# 78

df = pd.DataFrame({
"Employee":["A","B","C","D","E"],
"Dept":["IT","HR","IT","Finance","HR"],
"Salary":[50000,40000,70000,80000,45000]
})
df.groupby("Dept")
df
```

Out[74]:

	Employee	Dept	Salary
0	A	IT	50000
1	B	HR	40000
2	C	IT	70000
3	D	Finance	80000
4	E	HR	45000

In [75]:

```
df.groupby('Dept')['Salary'].agg(['sum','mean'])
```

Out[75]:

Dept	sum	mean
Finance	80000	80000.0
HR	85000	42500.0
IT	120000	60000.0

In [76]:
```
new=df.groupby('Dept')['Salary'].mean()
new[new>50000]
```

Out[76]:
```
Dept
Finance    80000.0
IT         60000.0
Name: Salary, dtype: float64
```

In [77]:
```
# 79

df = pd.DataFrame({
"Dept":["IT","IT","HR","HR","Finance","Finance"],
"Employee":["A","B","C","D","E","F"],
"Experience":[2,5,3,7,4,6]
})
new=df.groupby("Dept").size()
print(new)

df.sort_values("Experience", ascending=False).groupby("Dept").nth(0)
```

```
Dept
Finance    2
HR         2
IT         2
dtype: int64
```

Out[77]:

	Dept	Employee	Experience
3	HR	D	7
5	Finance	F	6
1	IT	B	5